

ViewModel State Management

50 Complete Examples with Different Patterns

Master state management in Android ViewModels: LiveData, StateFlow, sealed classes, derived states, form validation, pagination, authentication, shopping carts, media players, and more. Each example shows real-world patterns for managing and updating UI state.

BASIC STATE PATTERNS (1-10)

Example 1: Simple MutableLiveData State

```
class UserViewModel : ViewModel() { // Private mutable state private val _userName = MutableLiveData<String>() // Public immutable state val userName: LiveData<String> = _userName fun updateUserName(name: String) { _userName.value = name } } // In Activity/Fragment class MainActivity : AppCompatActivity() { private val viewModel: UserViewModel by viewModels() override fun onCreate(savedInstanceState: Bundle?) { super.onCreate(savedInstanceState) // Observe state changes viewModel.userName.observe(this) { name -> textView.text = name } // Update state viewModel.updateUserName("John Doe") } }
```

■ Basic pattern - LiveData for simple state management

Example 2: MutableStateFlow with StateFlow

```
class CounterViewModel : ViewModel() { private val _count = MutableStateFlow(0) val count: StateFlow<Int> = _count.asStateFlow() fun increment() { _count.value += 1 } fun decrement() { _count.value -= 1 } fun reset() { _count.value = 0 } } // In Composable or Fragment class CounterFragment : Fragment() { private val viewModel: CounterViewModel by viewModels() override fun onCreateView(view: View, savedInstanceState: Bundle?) { viewLifecycleOwner.lifecycleScope.launch { viewModel.count.collect { count -> countText.text = count.toString() } } plusButton.setOnClickListener { viewModel.increment() } minusButton.setOnClickListener { viewModel.decrement() } } }
```

■ Modern approach - StateFlow for reactive state updates

Example 3: Loading State Management

```
class ProductViewModel(private val repository: ProductRepository) : ViewModel() { private val _isLoading = MutableLiveData(false) val isLoading: LiveData<Boolean> = _isLoading private val _products = MutableLiveData<List<Product>>() val products: LiveData<List<Product>> = _products fun loadProducts() { viewModelScope.launch { _isLoading.value = true try { val result = repository.getProducts() _products.value = result } catch (e: Exception) { // Handle error } finally { _isLoading.value = false } } } } // In Activity viewModel.isLoading.observe(this) { loading -> progressBar.visibility = if (loading) View.VISIBLE else View.GONE } viewModel.products.observe(this) { products -> adapter.submitList(products) } }
```

■ Tracks loading state separately - common UI pattern

Example 4: Error State Handling

```
class LoginViewModel : ViewModel() { private val _email = MutableStateFlow("") val email: StateFlow<String> = _email private val _password = MutableStateFlow("") val password: StateFlow<String> = _password private val _errorMessage = MutableStateFlow<String?>(null) val errorMessage: StateFlow<String?> = _errorMessage fun updateEmail(email: String) { _email.value = email _errorMessage.value = null // Clear error } fun updatePassword(password: String) { _password.value = password _errorMessage.value = null } fun login() { viewModelScope.launch { if (_email.value.isBlank()) { _errorMessage.value = "Email is required" return@launch } if (_password.value.length < 6) { _errorMessage.value = "Password must be 6+ characters" return@launch } // Proceed with login... } } }
```

■ Validates input and manages error state

Example 5: Sealed Class UI State

```
sealed class UiState<out T> { object Loading : UiState<Nothing>() data class Success<T>(val data: T) : UiState<T>() data class Error(val message: String) : UiState<Nothing>() } class UserListViewModel : ViewModel() { private val _uiState = MutableStateFlow<UiState<List<User>>>(UiState.Loading) val uiState: StateFlow<UiState<List<User>>> = _uiState fun loadUsers() { viewModelScope.launch { _uiState.value = UiState.Loading try { val users = repository.getUsers() _uiState.value = UiState.Success(users) } catch (e: Exception) { _uiState.value = UiState.Error(e.message ?: "Unknown error") } } } } // In Fragment viewModel.uiState.collect { state -> when (state) { is UiState.Loading -> showLoading() is UiState.Success -> showUsers(state.data) is UiState.Error -> showError(state.message) } }
```

■ Type-safe state management - very robust pattern

Example 6: Multiple State Properties

```
data class ProfileState( val name: String = "", val email: String = "", val bio: String = "", val isEditing: Boolean = false, val isSaving: Boolean = false ) class ProfileViewModel : ViewModel() { private val _state = MutableStateFlow(ProfileState()) val state: StateFlow<ProfileState> = _state fun updateName(name: String) { _state.update { it.copy(name = name) } } fun updateEmail(email: String) { _state.update { it.copy(email = email) } } fun startEditing() { _state.update { it.copy(isEditing = true) } } fun saveProfile() { viewModelScope.launch { _state.update { it.copy(isSaving = true) } } // Save to repository... delay(1000) _state.update { it.copy(isSaving = false, isEditing = false) } } }
```

■ Data class for grouped state - cleaner, easier to manage

Example 7: List State Management

```
class TodoViewModel : ViewModel() { private val _todos = MutableStateFlow<List<Todo>>(emptyList()) val todos: StateFlow<List<Todo>> = _todos fun addTodo(title: String) { val newTodo = Todo(id = System.currentTimeMillis(), title = title) _todos.update { currentList -> currentList + newTodo } } fun removeTodo(id: Long) { _todos.update { currentList -> currentList.filter { it.id != id } } } fun toggleComplete(id: Long) { _todos.update { currentList -> currentList.map { todo -> if (todo.id == id) todo.copy(completed = !todo.completed) else todo } } } fun clearCompleted() { _todos.update { currentList -> currentList.filter { !it.completed } } } } data class Todo(val id: Long, val title: String, val completed: Boolean = false)
```

■ Immutable list updates - prevents bugs, thread-safe

Example 8: Derived State with Flow

```
class SearchViewModel : ViewModel() { private val _searchQuery = MutableStateFlow("") val searchQuery: StateFlow<String> = _searchQuery private val _allItems = MutableStateFlow<List<Item>>(emptyList()) // Derived state - automatically updates when dependencies change val filteredItems: StateFlow<List<Item>> = combine(_searchQuery, _allItems) { query, items -> if (query.isBlank()) items else items.filter { it.name.contains(query, ignoreCase = true) } }.stateIn( scope = viewModelScope, started = SharingStarted.WhileSubscribed(5000), initialValue = emptyList() ) fun updateSearchQuery(query: String) { _searchQuery.value = query } fun loadItems(items: List<Item>) { _allItems.value = items } }
```

■ Reactive derived state - no manual filtering needed

Example 9: Boolean Toggle States

```
class SettingsViewModel : ViewModel() { private val _darkMode = MutableStateFlow(false) val darkMode: StateFlow<Boolean> = _darkMode private val _notificationsEnabled = MutableStateFlow(true) val notificationsEnabled: StateFlow<Boolean> = _notificationsEnabled private val _autoSave = MutableStateFlow(true) val autoSave: StateFlow<Boolean> = _autoSave fun toggleDarkMode() { _darkMode.update { !it } // Save to preferences savePreference("dark_mode", _darkMode.value) } fun toggleNotifications() { _notificationsEnabled.update { !it } savePreference("notifications", _notificationsEnabled.value) } fun toggleAutoSave() { _autoSave.update { !it } savePreference("auto_save", _autoSave.value) } private fun savePreference(key: String, value: Boolean) { viewModelScope.launch { preferences.save(key, value) } } }
```

■ Toggle states for settings - clear boolean logic

Example 10: Nullable State Management

```
class UserProfileViewModel : ViewModel() { private val _user = MutableStateFlow<User?>(null) val user: StateFlow<User?> = _user private val _selectedImage = MutableStateFlow<Uri?>(null) val selectedImage: StateFlow<Uri?> = _selectedImage fun loadUser(userId: String) { viewModelScope.launch { val user = repository.getUser(userId) _user.value = user } } fun selectImage(uri: Uri) { _selectedImage.value = uri } fun clearImage() { _selectedImage.value = null } fun logout() { _user.value = null _selectedImage.value = null } } // In UI viewModel.user.collect { user -> if (user != null) { nameText.text = user.name emailText.text = user.email } else { // Show login screen } }
```

■ *Nullable states for optional data - null indicates absence*

INTERMEDIATE STATE (11-20)

Example 11: Pagination State

```
data class PaginationState<T>( val items: List<T> = emptyList(), val page: Int = 1, val isLoading: Boolean = false, val hasMorePages: Boolean = true, val error: String? = null ) class ProductListViewModel : ViewModel() { private val _state = MutableStateFlow<PaginationState<Product>()>() val state: StateFlow<PaginationState<Product>> = _state fun loadNextPage() { if (_state.value.isLoading || !_state.value.hasMorePages) return viewModelScope.launch { _state.update { it.copy(isLoading = true, error = null) } } try { val newItems = repository.getProducts(_state.value.page) _state.update { current -> current.copy( items = current.items + newItems, page = current.page + 1, isLoading = false, hasMorePages = newItems.isNotEmpty() ) } } catch (e: Exception) { _state.update { it.copy(isLoading = false, error = e.message) } } } } fun refresh() { _state.value = PaginationState() loadNextPage() } }
```

■ *Complete pagination state - tracks page, loading, errors*

Example 12: Form Validation State

```
data class FormField( val value: String = "", val error: String? = null, val isTouched: Boolean = false ) data class RegistrationFormState( val username: FormField = FormField(), val email: FormField = FormField(), val password: FormField = FormField(), val isValid: Boolean = false, val isSubmitting: Boolean = false ) class RegistrationViewModel : ViewModel() { private val _formState = MutableStateFlow<RegistrationFormState()>() val formState: StateFlow<RegistrationFormState> = _formState fun updateUsername(value: String) { _formState.update { state -> val error = when { value.length < 3 -> "Too short" value.length > 20 -> "Too long" else -> null } state.copy( username = FormField(value, error, true), isValid = validateForm(state.copy(username = FormField(value, error, true))) ) } } fun updateEmail(value: String) { _formState.update { state -> val error = if (!value.contains("@")) "Invalid email" else null state.copy( email = FormField(value, error, true), isValid = validateForm(state.copy(email = FormField(value, error, true))) ) } } private fun validateForm(state: RegistrationFormState): Boolean { return state.username.error == null && state.email.error == null && state.password.error == null && state.username.value.isNotBlank() } }
```

■ *Comprehensive form state - validation, errors, touched fields*

Example 13: Selection State (Multi-select)

```
class PhotoGalleryViewModel : ViewModel() { private val _photos = MutableStateFlow<List<Photo>>(emptyList()) val photos: StateFlow<List<Photo>> = _photos private val _selectedPhotoIds = MutableStateFlow<Set<Long>>(emptySet()) val selectedPhotoIds: StateFlow<Set<Long>> = _selectedPhotoIds val selectedCount: StateFlow<Int> = _selectedPhotoIds.map { it.size } .stateIn(viewModelScope, SharingStarted.Lazily, 0) val isSelectionMode: StateFlow<Boolean> = _selectedPhotoIds.map { it.isNotEmpty() } .stateIn(viewModelScope, SharingStarted.Lazily, false) fun togglePhotoSelection(photoId: Long) { _selectedPhotoIds.update { selected -> if (photoId in selected) selected - photoId else selected + photoId } } fun selectAll() { _selectedPhotoIds.value = _photos.value.map { it.id }.toSet() } fun clearSelection() { _selectedPhotoIds.value = emptySet() } fun deleteSelected() { viewModelScope.launch { _photos.update { photos -> photos.filter { it.id !in _selectedPhotoIds.value } } clearSelection() } } }
```

■ *Multi-selection state - common in galleries, file managers*

Example 14: Tab/Navigation State

```
enum class AppTab { HOME, SEARCH, PROFILE, SETTINGS } class MainViewModel : ViewModel() { private val _selectedTab = MutableStateFlow<AppTab>(AppTab.HOME) val selectedTab: StateFlow<AppTab> = _selectedTab private val _navigationHistory = MutableStateFlow<List<AppTab>>(listOf(AppTab.HOME)) val canGoBack: StateFlow<Boolean> = _navigationHistory.map { it.size > 1 } .stateIn(viewModelScope, SharingStarted.Lazily, false) fun selectTab(tab: AppTab) { if (_selectedTab.value != tab) { _selectedTab.value = tab _navigationHistory.update { it + tab } } } fun goBack(): Boolean { val history = _navigationHistory.value return if (history.size > 1) { val newHistory = history.dropLast(1) _navigationHistory.value = newHistory _selectedTab.value = newHistory.last() true } else { false } } fun resetNavigation() { _selectedTab.value = AppTab.HOME _navigationHistory.value = listOf(AppTab.HOME) } }
```

■ *Navigation state with history - supports back navigation*

Example 15: Filter State

```
data class FilterState( val category: String? = null, val minPrice: Double? = null, val maxPrice: Double? = null, val sortBy: SortOption = SortOption.NEWEST, val inStock: Boolean = false ) enum class SortOption { NEWEST, PRICE_LOW, PRICE_HIGH, POPULAR } class ProductFilterViewModel : ViewModel() { private val _filterState = MutableStateFlow<FilterState()>() val filterState: StateFlow<FilterState> = _filterState private val _allProducts = MutableStateFlow<List<Product>>(emptyList()) val filteredProducts: StateFlow<List<Product>> = combine( _filterState, _allProducts ) { filter, products -> products .filter { filter.category == null || it.category == filter.category } .filter { filter.minPrice == null || it.price >= filter.minPrice } .filter { filter.maxPrice == null || it.price <= filter.maxPrice } .filter { !filter.inStock || it.inStock } .let { filtered -> when (filter.sortBy) { SortOption.NEWEST -> filtered.sortedByDescending { it.createdAt } SortOption.PRICE_LOW -> filtered.sortedBy { it.price } SortOption.PRICE_HIGH -> filtered.sortedByDescending { it.price } SortOption.POPULAR -> filtered.sortedByDescending { it.rating } } } } .stateIn(viewModelScope, SharingStarted.Lazily, emptyList()) fun setCategory(category: String?) { _filterState.update { it.copy(category = category) } } fun setPriceRange(min: Double?, max: Double?) { _filterState.update { it.copy(minPrice = min, maxPrice = max) } } fun setSortOption(option: SortOption) { _filterState.update { it.copy(sortBy = option) } } fun clearFilters() { _filterState.value = FilterState() } }
```

■ *Complex filtering state - reactive filtering without manual updates*

Example 16: Timer/Countdown State

```
class TimerViewModel : ViewModel() { private val _timeRemaining = MutableStateFlow<Long>(0L) val timeRemaining: StateFlow<Long> = _timeRemaining private val _isRunning = MutableStateFlow<Boolean>(false) val isRunning: StateFlow<Boolean> = _isRunning private var timerJob: Job? = null fun startTimer(durationSeconds: Long) { timerJob?.cancel() _timeRemaining.value = durationSeconds _isRunning.value = true timerJob = viewModelScope.launch { while (_timeRemaining.value > 0) { delay(1000) _timeRemaining.update { it - 1 } } _isRunning.value = false onTimerComplete() } } fun pauseTimer() { timerJob?.cancel() _isRunning.value = false } fun resumeTimer() { if (_timeRemaining.value > 0) { startTimer(_timeRemaining.value) } } fun resetTimer() { timerJob?.cancel() _timeRemaining.value = 0 _isRunning.value = false } private fun onTimerComplete() { // Trigger notification or action } }
```

■ *Time-based state - useful for countdowns, timers, stopwatches*

Example 17: Network Connectivity State

```
enum class NetworkStatus { CONNECTED, DISCONNECTED, CONNECTING } class NetworkViewModel(private val connectivityManager: ConnectivityManager) : ViewModel() { private val _networkStatus = MutableStateFlow(NetworkStatus.DISCONNECTED) val networkStatus: StateFlow<NetworkStatus> = _networkStatus private val _isOnline = MutableStateFlow(false) val isOnline: StateFlow<Boolean> = _isOnline private val _pendingActions = MutableStateFlow<List<Action>>(emptyList()) val pendingActions: StateFlow<List<Action>> = _pendingActions init { observeNetworkChanges() } private fun observeNetworkChanges() { val networkCallback = object : ConnectivityManager.NetworkCallback() { override fun onAvailable(network: Network) { _networkStatus.value = NetworkStatus.CONNECTED _isOnline.value = true processPendingActions() } override fun onLost(network: Network) { _networkStatus.value = NetworkStatus.DISCONNECTED _isOnline.value = false } } connectivityManager.registerDefaultNetworkCallback(networkCallback) } fun queueAction(action: Action) { if (_isOnline.value) { executeAction(action) } else { _pendingActions.update { it + action } } } private fun processPendingActions() { viewModelScope.launch { _pendingActions.value.forEach { executeAction(it) } _pendingActions.value = emptyList() } } }
```

■ *Network state tracking - handles offline queue*

Example 18: Authentication State

```
sealed class AuthState { object Unauthenticated : AuthState() object Loading : AuthState() data class Authenticated(val user: User) : AuthState() data class Error(val message: String) : AuthState() } class AuthViewModel(private val authRepository: AuthRepository) : ViewModel() { private val _authState = MutableStateFlow<AuthState>(AuthState.Unauthenticated) val authState: StateFlow<AuthState> = _authState val isAuthenticated: StateFlow<Boolean> = _authState.map { it is AuthState.Authenticated }.stateIn(viewModelScope, SharingStarted.Eagerly, false) val currentUser: StateFlow<User?> = _authState.map { (it as? AuthState.Authenticated)?.user }.stateIn(viewModelScope, SharingStarted.Eagerly, null) fun login(email: String, password: String) { viewModelScope.launch { _authState.value = AuthState.Loading try { val user = authRepository.login(email, password) _authState.value = AuthState.Authenticated(user) } catch (e: Exception) { _authState.value = AuthState.Error(e.message ?: "Login failed") } } } fun logout() { viewModelScope.launch { authRepository.logout() _authState.value = AuthState.Unauthenticated } } fun checkAuthStatus() { viewModelScope.launch { val user = authRepository.getCurrentUser() _authState.value = if (user != null) { AuthState.Authenticated(user) } else { AuthState.Unauthenticated } } } }
```

■ *Complete auth state - tracks login status, user, errors*

Example 19: Shopping Cart State

```
data class CartItem(val product: Product, val quantity: Int) data class CartState( val items: List<CartItem> = emptyList(), val subtotal: Double = 0.0, val tax: Double = 0.0, val shipping: Double = 0.0, val total: Double = 0.0, val itemCount: Int = 0 ) class ShoppingCartViewModel : ViewModel() { private val _cartState = MutableStateFlow<CartState>() val cartState: StateFlow<CartState> = _cartState fun addToCart(product: Product, quantity: Int = 1) { _cartState.update { state -> val existingItem = state.items.find { it.product.id == product.id } val newItem = if (existingItem != null) { state.items.map { item -> if (item.product.id == product.id) { item.copy(quantity = item.quantity + quantity) } else item } } else { state.items + CartItem(product, quantity) } calculateTotals(newItems) } } fun removeFromCart(productId: Long) { _cartState.update { state -> val newItems = state.items.filter { it.product.id != productId } calculateTotals(newItems) } } fun updateQuantity(productId: Long, quantity: Int) { if (quantity <= 0) { removeFromCart(productId) return } _cartState.update { state -> val newItems = state.items.map { item -> if (item.product.id == productId) item.copy(quantity = quantity) else item } calculateTotals(newItems) } } private fun calculateTotals(items: List<CartItem>): CartState { val subtotal = items.sumOf { it.product.price * it.quantity } val tax = subtotal * 0.08 val shipping = if (subtotal > 50) 0.0 else 5.99 val total = subtotal + tax + shipping val itemCount = items.sumOf { it.quantity } return CartState(items, subtotal, tax, shipping, total, itemCount) } fun clearCart() { _cartState.value = CartState() } }
```

■ *E-commerce cart state - handles items, quantities, totals*

Example 20: Media Player State

```
data class MediaState( val currentTrack: Track? = null, val isPlaying: Boolean = false, val currentPosition: Long = 0, val duration: Long = 0, val playbackSpeed: Float = 1.0f, val volume: Float = 1.0f, val repeatMode: RepeatMode = RepeatMode.OFF, val shuffleEnabled: Boolean = false ) enum class RepeatMode { OFF, ONE, ALL } class MediaPlayerViewModel : ViewModel() { private val _mediaState = MutableStateFlow<MediaState>() val mediaState: StateFlow<MediaState> = _mediaState val progress: StateFlow<Float> = _mediaState.map { state -> if (state.duration > 0) state.currentPosition.toFloat() / state.duration else 0f }.stateIn(viewModelScope, SharingStarted.Lazily, 0f) fun playTrack(track: Track) { _mediaState.update { it.copy(currentTrack = track, isPlaying = true, currentPosition = 0) } startPositionUpdates() } fun togglePlayPause() { _mediaState.update { it.copy(isPlaying = !it.isPlaying) } } fun seekTo(position: Long) { _mediaState.update { it.copy(currentPosition = position) } } fun setPlaybackSpeed(speed: Float) { _mediaState.update { it.copy(playbackSpeed = speed) } } fun toggleRepeatMode() { _mediaState.update { state -> val newMode = when (state.repeatMode) { RepeatMode.OFF -> RepeatMode.ONE RepeatMode.ONE -> RepeatMode.ALL RepeatMode.ALL -> RepeatMode.OFF } state.copy(repeatMode = newMode) } } private fun startPositionUpdates() { viewModelScope.launch { while (_mediaState.value.isPlaying) { delay(100) _mediaState.update { state -> if (state.currentPosition < state.duration) { state.copy(currentPosition = state.currentPosition + 100) } else { handleTrackEnd(state) } } } } }
```

■ *Complex media state - playback control, position, settings*